Team Workspace Website — Full Build Specification

1. What this project is

A website where small teams can register, form a team under one leader, and share files and announcements in a private team workspace. Three main areas: **Landing Page**, **Authentication (Register/Login)**, and **Workspace**.

---

2. User Roles

There are two roles:

- **Leader** — the person who creates a team. A leader can:
  - Add members to the team by their User ID
  - Remove members from the team
  - Post announcements with deadlines
  - Upload, view, and delete any file in the team
  - Rename the team or edit the project idea
- **Member** — anyone added to a team by a leader. A member can:
  - View announcements
  - Upload and view files
  - Delete only the files they personally uploaded
  - Leave the team at any time

Rule: A user can belong to MULTIPLE teams at the same time, in any mix of roles — for example, a user can be the Leader of one team and a Member of another, or a Member of several teams at once. Each team's announcements and files stay completely separate from every other team the user is part of.

---

3. Account System

3.1 Registration

1. User goes to the Register page.
2. User enters: **email address** and **password** (password must be at least 8 characters).
3. **Rule:** Each email address can be used to create only ONE account. If someone tries to register with an email that's already registered, show a clear error: "An account with this email already exists. Please log in instead." Do not create a second account.
4. System sends a verification link to that email. The account is not active until the user clicks the link.

5. Once verified, the system automatically generates a **User ID** in this exact format: `NNNLLNN` — 3 numbers, then 2 uppercase letters, then 2 numbers (example: `304TX58`).
6. The system checks the new ID against all existing IDs. If it already exists, the system generates a new one automatically and checks again, until it finds one that is not used. The user never sees this retry — they only see the final, unique ID.
7. The User ID is shown on screen once and also emailed to the user. It cannot be changed later.

3.2 Login

1. User enters their **User ID** and their **password**.
2. If both match a registered, verified account, user enters the Workspace.
3. If the User ID doesn't exist, show: "No account found with this User ID."
4. If the User ID exists but the password is wrong, show: "Incorrect password." (Don't reveal whether it was the ID or the password that was wrong for a security reason — but for this small-team project, a straightforward message is fine.)
5. Include two separate recovery links, since they solve different problems:
    - **"Forgot Password"** — user enters their email; system sends a password reset link if that email matches a registered account. This does NOT reveal or need the User ID.
    - **"Forgot User ID"** — user enters their email; system emails back their User ID if that email matches a registered account. This does NOT reset or need the password.

---

4. Workspace — First Screen

After login, show a **"My Teams"** list with every team the user currently belongs to (each shown with its name and the user's role in it — Leader or Member). Clicking a team opens that team's Team Workspace (Section 6).

Two buttons are always visible on this screen, no matter how many teams the user already has:

- **Create a Team**
- **Join a Team**

If the user has no teams yet, the "My Teams" list is simply empty and only the two buttons show.

---

5. Creating or Joining a Team

5.1 Create a Team

1. User (this person automatically becomes the **Leader**) fills a form with:
    - Team Name (text)

- Project Idea (short text description)
- **Invited Members (optional):** one or more boxes where the Leader can type the **User ID** of each person they want to allow into the team. This step can be skipped and people invited later (see Section 5.2).

2. On submit, the system:
   - Creates the team
   - Sets this user as Leader
   - For each User ID typed in the Invited Members boxes, runs the same checks as Section 5.2 (ID must exist; show an error next to any ID that's invalid, without stopping the rest of the form) and adds each valid one to the team's **Invited List** with status "Pending" (not yet an active Member)
   - Generates a **Team Code** in this exact format: `NNNLL` — 3 numbers then 2 uppercase letters (example: `582QK`).
   - Checks the code isn't already used by another team; if it is, generates a new one and checks again, same as the User ID process.
3. The Team Code is shown to the Leader on screen so they can share it with the people they invited.

5.2 Invite a Member by User ID (Leader only)

1. Leader enters a teammate's **User ID** in a text box and submits.
2. The system checks:
   - **Does this User ID exist?** If not → show error: "No user found with this ID. Please check and try again." Do nothing else.
   - **Is this user already on the Invited List or an active Member of THIS SAME team?** If yes → show error: "This user is already invited or already on this team." Do nothing else.
   - Otherwise → add that User ID to this team's **Invited List** with status "Pending." This does NOT make them an active Member yet — they still need to enter the Team Code (Section 5.3) to activate their membership. It does not matter if that user is already on other teams.

5.3 Join a Team by Team Code (only works for User IDs the Leader already invited)

1. User enters a **Team Code** in a text box and submits.
2. The system checks:
   - **Does this Team Code exist?** If not → show error: "No team found with this code. Please check and try again."

- **Is this user's own User ID on that team's Invited List?** If not → show error: "You have not been invited to this team. Ask the team leader to add your User ID first." Do NOT add them, even though the code was correct.
- **Is the user already an active Member of THIS SAME team?** If yes → show error: "You are already on this team."
- Otherwise → change this user's status on the Invited List from "Pending" to active **Member**.

In short: the Team Code by itself is never enough to join. It only works for a User ID the Leader put on the Invited List (either at team creation, Section 5.1, or afterward, Section 5.2). Anyone else entering the code, even the correct one, is rejected.

---

6. Team Workspace (main working screen)

Once inside a team, every member (Leader and Members) sees four sections on one page, plus a "Back to My Teams" link/button always visible (see Section 4) so the user can switch to a different team they belong to.

6.1 Team Info Bar (top of page)

- Team Name
- Project Idea
- Team Code (visible only to the Leader, with a "copy" button)
- List of people on the team, showing each one's status: **Pending** (invited but hasn't entered the Team Code yet) or **Member** (active)
- **Manage Members (Leader only):** a button next to the list that opens a small panel where the Leader can:
  - Invite a new person by typing their User ID (same check rules as Section 5.2) — they'll show as "Pending" until they enter the Team Code
  - Remove any person, Pending or active Member (removing does not delete that person's account — it only takes them off this team's list)

6.2 Announcements

- A simple feed, newest at the top.
- Each announcement has: message text, an optional deadline date, and who posted it and when.
- Only the Leader sees a box to write a new announcement.
- Members see the feed but cannot post.

6.3 Team Chat

- One shared chat for the whole team — not private one-to-one messages between individual members.
- Any team member (Leader or Member) can post a message.
- Each message shows: the message text, who sent it, and the time it was sent.
- Messages appear in order, oldest at the top, newest at the bottom.
- The chat updates when the page is opened or refreshed — it does not need to update live/instantly while the page is sitting open, to keep this simple and low-cost to build.
- No message can be edited or deleted once sent, to keep this section simple.

6.4 Shared Files

- A list of uploaded files, each showing: file name, who uploaded it, upload date, and a download button.
- Any team member can upload a file.
- Delete button appears only next to: files the current user uploaded themselves, OR any file if the current user is the Leader.
- No file-size limit needs to be enforced manually — use whatever limit the hosting platform's free storage tier provides.

6.5 Leaving a Team

- A "Leave Team" button is available to Members at all times. Leaving only removes the user from this one team — their membership in any other teams is unaffected.
- If the Leader wants to leave, the system must first ask: "You are the leader. You must either delete the whole team or assign a new leader before leaving." (Do not allow a team to exist with no leader.)

6.6 Personal Profile Settings (per-user, does not affect teammates)

This is personal to each individual user, not something a Leader controls or other members can see change — it only affects how the app looks/feels for that one person.

- **Theme Color:** exactly these 7 preset swatches to pick from — Black, White, Yellow, Blue, Green, Pink, Brown. Whatever they pick changes the accent color of buttons/highlights only in their own browser — it does not change what anyone else on their team sees.
- **Personal Motivational Quote:** a short text box where the user can type their own quote. It's shown back to them (e.g. on their "My Teams" screen) as a small personal reminder. Optional — if left blank, nothing shows.
- Both settings are saved to that user's own account and stay the same every time they log back in, on any device.

6.7 Personal To-Do List (per-user, private — not the same as Team Announcements)

This is separate from Section 6.2 Announcements. Announcements are posted by the Leader for the whole team to read; this To-Do List is each individual user's own private checklist, visible only to them, for tracking work the team assigned to them personally so they don't forget it.

- Visible right after login — a "My To-Do List" section on the My Teams screen (Section 4), so it's one of the first things the user sees.
- The user can:
  - Add a new task (short text describing the work, e.g. "Finish the login page design")
  - Optionally pick a due date for that task
  - Optionally tag which team the task relates to, if they're in more than one team (helps them remember which project it's for) — this is just a label for their own reference, not visible to anyone else
  - Mark a task as done (it can move to a "Completed" section, or just get a checkmark/strikethrough)
  - Delete a task at any time
- No one else — not even the Leader — can see or edit another person's to-do list. It exists purely to help that one person stay on top of their own work.
- Nothing here is linked to Announcements automatically; if a Leader posts an announcement, the user has to manually add it to their own to-do list if they want a reminder — keeping this simple rather than trying to auto-sync the two.

6.8 AI Assistant (separate from Team Chat)

A standalone chat window any logged-in user can open to ask general questions — completely separate from Section 6.3 Team Chat. This assistant does NOT see or use any team data (files, announcements, other chat messages) — it's just a general-purpose Q&A helper, kept simple and private to the one user asking.

- A chat icon/button, reachable from anywhere in the Workspace (not tied to any specific team).
- The user types a question, the assistant (powered by Grok) replies in the same window.
- Conversation history is private to that user — not visible to teammates or the Leader.
- **Cost note:** unlike the rest of this app, this feature has a real ongoing cost based on usage, since the AI provider charges per question asked. Keep an eye on usage if it gets used a lot.

---

7. Error Handling Rules (apply everywhere)

- Every error message must say exactly what went wrong in plain words — never a blank screen or a generic "something went wrong."
- No action should ever partially complete. Example: if adding a member fails, the team must remain exactly as it was before — nothing is added, changed, or removed.
- All ID and code checks happen instantly when the user submits the form, not after a delay.

---

8. Build Order (do these steps one at a time, and test each before moving to the next)

1. Landing page with Login and Register buttons.
2. Register flow, including email verification and automatic User ID generation with the duplicate-check rule.
3. Login flow (User ID + password), including both "Forgot Password" and "Forgot User ID."
4. "My Teams" list screen with "Create a Team" / "Join a Team" buttons.
5. Create Team flow, including the optional Invited Members boxes and Team Code generation with the duplicate-check rule.
6. Invite/Remove Member by User ID flow (used both during creation and later from the Manage Members panel), including both error cases (ID not found, already invited/already a member).
7. Join Team by Team Code flow, activating Pending invites into active Members, including all three error cases (code not found, not on the Invited List, already an active member).
8. Team Workspace screen: Team Info Bar, member list with Pending/Member status, Manage Members panel, and "Back to My Teams" switcher.
9. Announcements section (post for Leader, view for everyone).
10. Team Chat section (any member can post, refresh-to-update, no edit/delete).
11. Shared Files section (upload for everyone, delete rules as described).
12. Leave Team button, including the special rule for a Leader.
13. Test the entire sequence end-to-end using at least 3 separate test accounts before considering it finished.

---

9. Technical Notes for the Builder

- Use a real database (not a spreadsheet or static file) because User IDs, Team Codes, team membership, and files all need to stay in sync with no duplicates.
- Store passwords encrypted (never as plain text) — any standard authentication system handles this automatically.

- Team file storage must be private: a user who is not a member of a team must never be able to see or download that team's files, even with a direct link.
- The `profiles` table needs two extra columns for Section 6.6: `theme_color` (text, one of the preset options, defaults to the standard color if unset) and `personal_quote` (text, optional, can be blank).
- A new table is needed for Section 6.7: `personal_todos` (columns: id, user_id, task_text, due_date optional, linked_team_id optional, is_done boolean, created_at). This table must be locked down so a user can only ever read/write their own rows — never another user's tasks, including the Leader.
- For Section 6.8, the Grok API key must NEVER be placed in frontend code, where anyone could see and misuse it. It needs a small serverless function (e.g. a Netlify Function, since that's already part of this project's hosting) that holds the key privately and forwards the user's question to Grok, then returns the reply. A new table `ai_chat_history` (columns: id, user_id, message, sender ("user" or "assistant"), created_at) stores each user's own conversation, locked down the same way as `personal_todos` — a user can only read their own rows.